AUTOBIZ AI

An AI-Based Multi-Agent Website Generator for Small Businesses

PROJECT REPORT

Django Edition — a port of the CodeIgniter 4 / PHP application to Python, Django and MySQL

Technology: Python 3.13 · Django 6.1 · MariaDB / MySQL · OpenAI API · Tailwind CSS
Project folder: C:\D Drive\website_agent_django
Date: 29 October 2026

ABSTRACT

Small businesses such as restaurants, clinics, salons and agencies often have no website, or an out-dated one, because a professional design costs time and money. This project presents **AutoBiz AI**, a web application that turns the basic details of a business — its name, type, city and contact information — into five complete, professionally styled single-page websites. A chain of cooperating software agents does the work: an Enrichment agent uses a large language model (OpenAI GPT-4o) to build a brand profile, a Strategy agent writes the website copy, a Design agent generates one full HTML page for each of five visual styles, and a Build agent saves the pages so the operator can preview them in the browser or download them as a ZIP file. Hosting (FTP) and Outreach (e-mail) agents are implemented for later use.

The system was first built with PHP and CodeIgniter 4. This report documents its re-implementation in **Python with the Django framework and a MySQL/MariaDB database**. The port keeps the database schema, the agent prompts and the user interface unchanged, adds a one-click start script, a legacy data importer, an administration panel and an automated test suite, and fixes defects found during the analysis. All 14 automated tests pass against the target MariaDB 10.4 server, the five design styles were verified to be byte-for-byte identical to the original, and the existing business data was migrated successfully.

Keywords: Website Generation, Large Language Models, Multi-Agent Systems, Prompt Engineering, Python, Django, MySQL, OpenAI API, Small Business Automation, Software Migration

INDEX

CHAPTER NO.	TOPIC	P.NO
01	**INTRODUCTION** 1.1 Objective of the Project 1.2 Scope of the Project	1
02	**LITERATURE SURVEY**	2-3
03	**SYSTEM STUDY ANALYSIS** 3.1 Existing System 3.2 Proposed System 3.3 Modules 3.4 Feasibility Analysis	4-5
04	**REQUIREMENT ANALYSIS AND SPECIFICATION** 4.1 Software Requirement Specification 4.2 Hardware and Software Requirements	6-7
05	**SYSTEM DESIGN** 5.1 UML Diagrams 5.2 Database Design	8-14
06	**SYSTEM IMPLEMENTATION** 6.1 Python and Django 6.2 Algorithms 6.3 Sample Code	15-16
07	**SYSTEM TESTING** 7.1 Types of Testing 7.2 Test Cases	17-18
08	**SCREENSHOTS**	19-22
09	**CONCLUSION**	23
10	**FUTURE ENHANCEMENTS**	24
11	**REFERENCES**	25

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	P.NO
3.4.1	SDLC Stages Followed in the Project	5
5.1.1	Class Diagram	8
5.1.2	Use Case Diagram	9
5.1.3	Sequence Diagram	10
5.1.4	Collaboration Diagram	11
5.1.5	Component Diagram	12
5.1.6	Deployment Diagram	12
5.1.7	Activity Diagram	13
5.1.8	Data Flow Diagram	14
8.1	Dashboard	19
8.2	All Businesses	20
8.3	Add Business Form	20
8.4	Form Validation Errors	21
8.5	Business Detail	21
8.6	Generated Design — Prestige Style	22
8.7	Generated Design — Velocity Style	22

CHAPTER - 1
INTRODUCTION

A website is now the first place customers look for a business: to read what it offers, to find its address and opening hours, and to decide whether to call or visit. Many small businesses still have none. A custom website from a design agency takes days of work — a brief, copywriting, visual design, HTML and CSS development and revisions — and its cost is out of reach for a neighbourhood restaurant or a single-doctor clinic. Template-based site builders are cheaper, but the owner still has to write the content and assemble the pages, and the results tend to look alike.

Large language models (LLMs) changed what can be automated in this process. A model such as GPT-4o can analyse a business from a few facts, write persuasive marketing copy in a chosen tone, and produce complete HTML and CSS for a page when it is given precise instructions. AutoBiz AI uses this ability in a controlled way: instead of a single open-ended request, the work is split into small, well-defined steps, each handled by a dedicated agent with its own prompt and its own stored result. The output of one step becomes the input of the next, and every step can be inspected and repeated.

An operator (for example a freelancer or a small agency) adds a business through a web dashboard. The pipeline then enriches the business with a brand profile, writes a content strategy, generates five complete website designs in five distinct visual styles, and writes them to disk. The operator previews the designs, downloads them as a ZIP file and presents them to the business owner. When the business already has a website, the Design agent fetches it first and produces redesigns that reuse its real content.

The first version of AutoBiz AI was written in PHP with the CodeIgniter 4 framework. This project re-implements the complete application in Python with the Django framework, keeping the behaviour and data of the original while improving its structure, testability and ease of setup.

1.1 Objective of the Project

The objectives of the project are:

1. To generate complete, responsive single-page websites for a business automatically from a small set of facts, using a pipeline of specialised LLM agents.
2. To give the operator a dashboard to onboard businesses, run the pipeline, follow its progress, preview the generated designs and download them.
3. To port the existing CodeIgniter 4 / PHP application to Python and Django without changing its database schema, prompts or user interface, so existing data can be migrated unchanged.
4. To make the system easy to install and verify: a one-click start script, an offline mode that needs no API key, and an automated test suite.

1.2 Scope of the Project

The project covers the full web application: the five database tables, the seven services (OpenAI client, Enrichment, Strategy, Design, Build, Hosting and Outreach agents) and the Orchestrator that runs them, ten web routes with four screens, a command-line pipeline runner, a data importer for the original database and an administration panel. The Hosting and Outreach agents are implemented but switched off in the pipeline, as in the original, because generated designs are delivered as a ZIP download. Automatic discovery of businesses (web scraping) is outside the current scope.

CHAPTER - 2
LITERATURE SURVEY

The project draws on research in large language models, automatic code generation and multi-agent LLM systems. The works below informed the design of the agent pipeline.

"Language Models are Few-Shot Learners"

T. Brown et al., 2020.

This paper introduced GPT-3, a 175-billion-parameter language model, and showed that a sufficiently large model can perform new tasks from a natural-language instruction and a few examples, without task-specific training. This "in-context learning" is the basis of AutoBiz AI: every agent describes its task, the expected output format and the business data in the prompt, and relies on the model to follow the instructions.

"Evaluating Large Language Models Trained on Code"

M. Chen et al., 2021.

The authors present Codex, a GPT model fine-tuned on public source code, and the HumanEval benchmark for measuring whether generated programs are functionally correct. The work showed that language models can write working code from a description, and that sampling and checking outputs improves reliability — the reason AutoBiz AI validates every generated page (it must contain an HTML document) before storing it.

"pix2code: Generating Code from a Graphical User Interface Screenshot"

T. Beltramelli, 2017.

pix2code trains a neural network to translate a screenshot of a user interface into code for web, Android and iOS targets. It was an early demonstration that front-end code can be produced automatically from a visual specification. AutoBiz AI approaches the same goal from text: the visual specification is a written page blueprint and a CSS foundation instead of an image.

"Design2Code: How Far Are We From Automating Front-End Engineering?"

C. Si et al., 2024.

Design2Code builds a benchmark of real web pages and measures how closely multimodal LLMs can reproduce them as HTML and CSS. The study reports that current models already produce usable pages and that detailed prompting improves layout fidelity. This supports the design choice in AutoBiz AI of giving the model a section-by-section blueprint and a fixed set of CSS variables instead of a short request.

"ReAct: Synergizing Reasoning and Acting in Language Models"

S. Yao et al., 2023.

ReAct interleaves model reasoning with actions such as tool calls, so that each step can use information gathered in the previous one. AutoBiz AI applies a simpler, deterministic form of this idea: the Orchestrator fixes the order of steps, and each agent acts on the stored results of the earlier ones.

"ChatDev: Communicative Agents for Software Development"

C. Qian et al., 2023.

ChatDev organises several LLM agents with roles such as designer, programmer and tester into a virtual software company that follows a waterfall-style process. The paper shows that dividing a large generation task among role-specific agents improves completeness. AutoBiz AI follows the same principle with roles for brand analysis, copywriting, design and build.

"MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework"
S. Hong et al., 2023.

MetaGPT encodes standard operating procedures into a multi-agent framework, with each agent producing a structured document for the next. Structured hand-offs reduce errors that build up across steps. In AutoBiz AI the hand-offs are JSON objects (brand profile, content strategy) stored in the database, which makes each step repeatable and easy to inspect.

Summary. The literature shows that LLMs can write convincing copy and working front-end code, and that splitting a large task among specialised agents with structured hand-offs makes the result more complete and reliable. AutoBiz AI combines these findings into a practical tool for one domain — websites for small businesses — with fixed visual styles to keep the output consistent.

CHAPTER - 3
SYSTEM STUDY ANALYSIS

3.1 Existing System

Without automation, a website for a small business is produced manually: an agency collects information from the owner, a copywriter drafts the text, a designer prepares mock-ups and a developer builds the pages. Each design option multiplies the effort, so owners usually see only one or two proposals.

The first automated version of AutoBiz AI was written in PHP 8 on CodeIgniter 4.4, served by Apache from XAMPP with a MariaDB 10.4 database. It introduced the agent pipeline, but had several weaknesses that were found while analysing it for the port:

Disadvantages:

1. No automated tests; every change had to be checked by hand.
2. Without an OpenAI key the stub replies returned HTML to the Strategy agent, so the offline pipeline always stopped at the second step.
3. The manual status endpoint wrote any submitted value to the database.
4. Setup required configuring Apache virtual hosts and running migrations by hand.
5. Interface text promised ten designs although five styles exist; the outreach e-mail had a hard-coded reply address.

3.2 Proposed System

The proposed system is the Django edition of AutoBiz AI. It keeps the five tables with identical names, the seven services, the five design styles and prompts, and the four screens, and re-implements them in Python. The pipeline produces five complete designs for a business in a few minutes, so an owner can compare several finished options instead of mock-ups.

Advantages:

1. Five complete, responsive designs per business with no manual design work.
2. One-click setup and start with run.bat; works with MySQL, MariaDB or SQLite.
3. 14 automated tests and an offline mode that runs the whole pipeline without API cost.
4. Existing data migrates unchanged through the import_legacy command.
5. Input validation, CSRF protection and POST-only state changes from the Django framework.

3.3 Modules

Module	Description
Dashboard	Overview page with a count of businesses per pipeline stage, conversion, contact and deploy rates, the agent pipeline and the eight most recent businesses.
Business management	List with stage filter and pagination, onboarding form with validation, detail page, manual status change and delete (rows and build files).
Enrichment agent	Sends the business facts to GPT-4o and stores a brand profile: summary, target audience, branding notes, unique selling proposition, tone, colour palette, keywords.
Strategy agent	Writes the website copy as JSON: hero headline and sub-headline, about text, 3–4 services, call to action, footer tagline and SEO page title.

Module	Description
Design agent	For each of the five styles, builds a detailed prompt and asks the model for a complete HTML page; fetches an existing website first to switch to redesign mode.
Build agent	Writes every draft design to writable/builds/<business>/v<N>_<style>.html and marks it built.
Preview and download	Serves a stored design as a web page and packs all built files into a ZIP file.
Hosting agent	Uploads built pages by FTP and records demo URLs (implemented, disabled).
Outreach agent	Sends an HTML e-mail with demo links to the owner by SMTP (implemented, disabled).
Orchestrator and CLI	Runs the agents in order for one business or for all pending ones; available as manage.py agent_run and run_pipeline.bat.
Data migration and admin	import_legacy copies all data from the original database; the Django admin gives direct access to all five tables.

3.4 Feasibility Analysis

Technical feasibility. The system uses mature, free technologies — Python, Django, PyMySQL and MariaDB — and a commercial API that is reachable over HTTPS. It runs on an ordinary Windows PC with XAMPP. The only non-standard requirement, Django 6 on MariaDB 10.4, was solved with a small database backend and verified by the test suite.

Economic feasibility. All software used is open source. The running cost is the OpenAI usage: seven model calls per business (one enrichment, one strategy and five designs). This is small compared with the cost of designing five websites by hand, and the offline mode allows development and demonstrations at no cost.

Operational feasibility. The operator works in a familiar web dashboard; starting the system is a double-click. The pipeline shows a progress overlay while it runs, and every result can be previewed, regenerated or deleted.

Software development life cycle. The project followed the stages shown in Fig 3.4.1, adapted to a migration project: the requirements were derived from the existing code, and every stage was checked against the behaviour of the original.

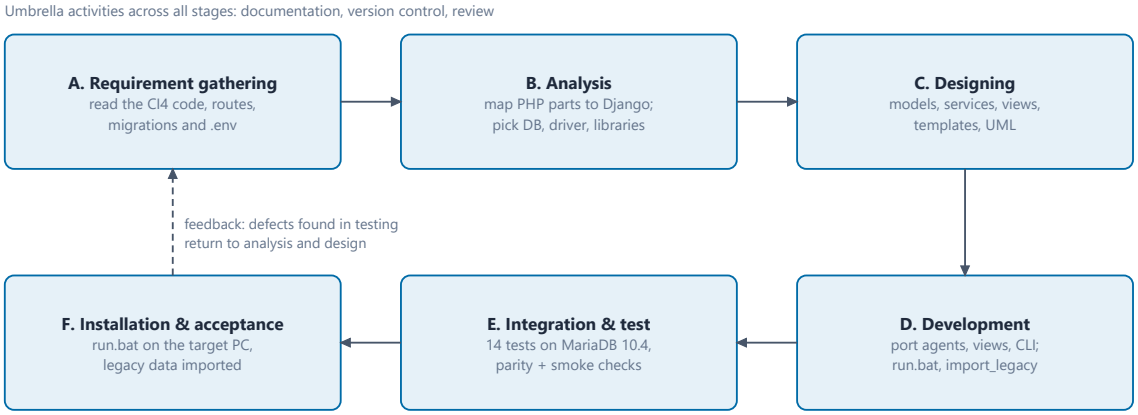


Fig 3.4.1 SDLC Stages Followed in the Project

CHAPTER - 4
REQUIREMENT ANALYSIS AND SPECIFICATION

4.1 Software Requirement Specification

A Software Requirements Specification (SRS) is a complete description of the behaviour of the system to be developed. It lists the functional requirements — what the system must do — and the non-functional requirements that constrain how it does it.

4.1.1 Overall Description

AutoBiz AI is a stand-alone web application used by one operator role. It runs on the operator's computer or on a server, stores its data in a MySQL-compatible database, and calls the OpenAI API to generate content. The operator needs only a web browser. The system must stay compatible with the database schema of the original application.

4.1.2 Functional Requirements

ID	Requirement
FR-01	Add a business with name, type, category, location, address, phone, e-mail, website status, website URL, rating and review count; validate the input.
FR-02	Run the pipeline for one business from the dashboard ("Run", "Save & Run").
FR-03	Generate a brand profile, a content strategy and one complete HTML design per style.
FR-04	Redesign mode: fetch an existing website and use its title, theme colour and text.
FR-05	Save built designs to disk; preview any design; download all designs as a ZIP file.
FR-06	Regenerate designs: delete the old designs and files and run strategy, design and build again.
FR-07	List businesses with stage filter and pagination; show a dashboard with stage counts and rates.
FR-08	Change a business status by hand (only the seven valid values); delete a business and its data.
FR-09	Run the pipeline from the command line for all pending businesses or for one id.
FR-10	Import all data from the original CodeIgniter database.
FR-11	Deploy designs by FTP and send outreach e-mail (available, disabled by default).

4.1.3 Non-Functional Requirements

- **Performance:** a full pipeline run (seven model calls) completes in about 1–3 minutes; dashboard pages render in well under a second.
- **Security:** CSRF protection on every form; state changes only through POST; validated input; secrets kept in a .env file outside version control; the server binds to the local machine by default.
- **Reliability:** a failed step marks the business "skipped" instead of failing the request; a style whose reply is not HTML is skipped while the others continue.
- **Maintainability:** one module per agent, settings in one place, automated tests for the pipeline and every route.
- **Portability:** runs on Windows, Linux or macOS with Python 3.12+; supports MySQL, MariaDB and SQLite.

4.1.4 External Interfaces

- **User interface:** web dashboard built with Django templates and Tailwind CSS.
- **Software interfaces:** OpenAI Chat Completions API over HTTPS; MySQL/MariaDB over TCP 3306; SMTP and FTP for the optional agents.
- **Operating environment:** Windows 10/11 with XAMPP (MariaDB) and Python, or any server with Python and MySQL.

4.2 Hardware and Software Requirements

A. Hardware Requirements:

Component	Minimum
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended)
Hard disk	1 GB free for the application, database and builds
Network	Internet connection for the OpenAI API

B. Software Requirements:

Software	Version / purpose
Operating system	Windows 10/11 (tested), Linux or macOS
Programming language	Python 3.13 (3.12 or newer)
Framework	Django 6.1.2
Database	MariaDB 10.4+ (XAMPP) or MySQL 8; SQLite for demonstrations
Libraries	PyMySQL 1.2.3, requests 2.34, python-dotenv 1.2
AI service	OpenAI API key (model gpt-4o); optional — offline stub without it
Browser	Any current browser (Chrome, Edge, Firefox)

CHAPTER - 5
SYSTEM DESIGN

5.1 UML Diagrams

The Unified Modelling Language (UML) is a standard language for specifying, visualising and documenting the structure and behaviour of a software system. The structural diagrams below (class, component and deployment) describe how AutoBiz AI is organised; the behavioural diagrams (use case, sequence, collaboration and activity) and the data flow diagram describe how it works when the pipeline runs.

a) Class Diagram

The class diagram shows the five persistent classes of the domain model, their attributes and their associations. A Business owns its enrichments, designs and outreach e-mails, and every design can have FTP deployments. All associations are compositions: deleting a business deletes everything that belongs to it.

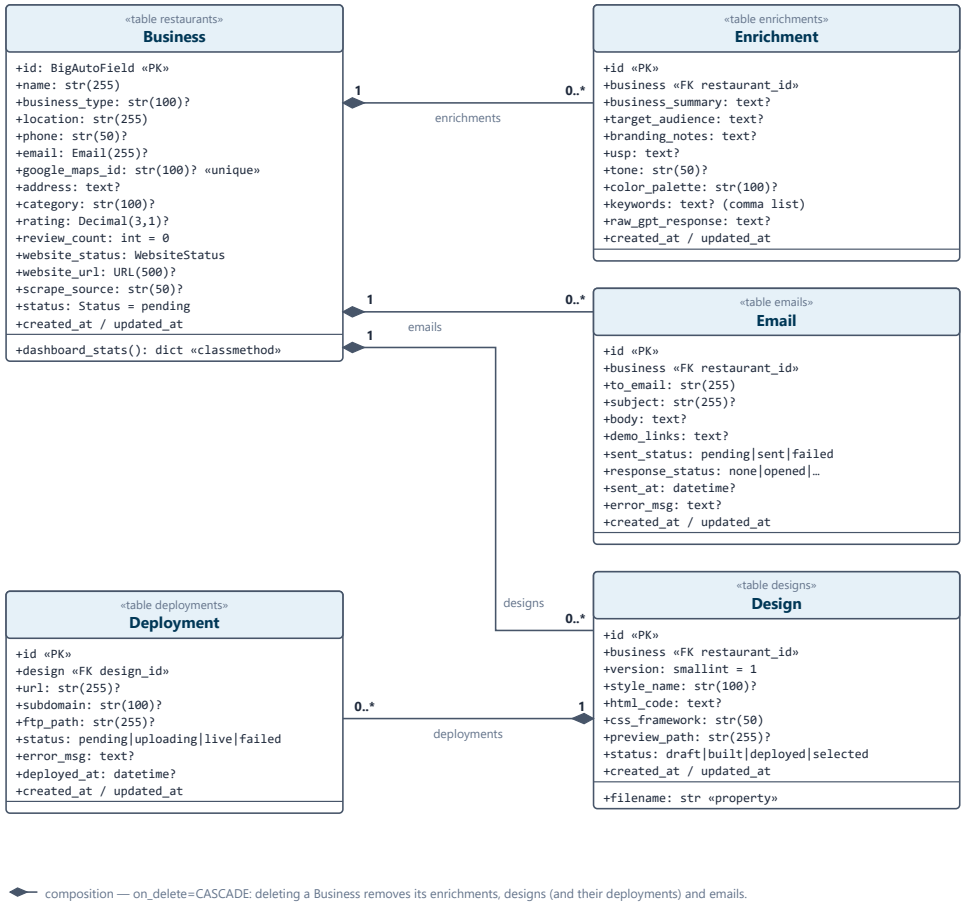


Fig 5.1.1 Class Diagram

b) Use Case Diagram

The use case diagram shows how the actors interact with the system. The operator uses the dashboard for all tasks; a command-line user or scheduler processes all pending businesses; the OpenAI API is the external system used by every pipeline run.

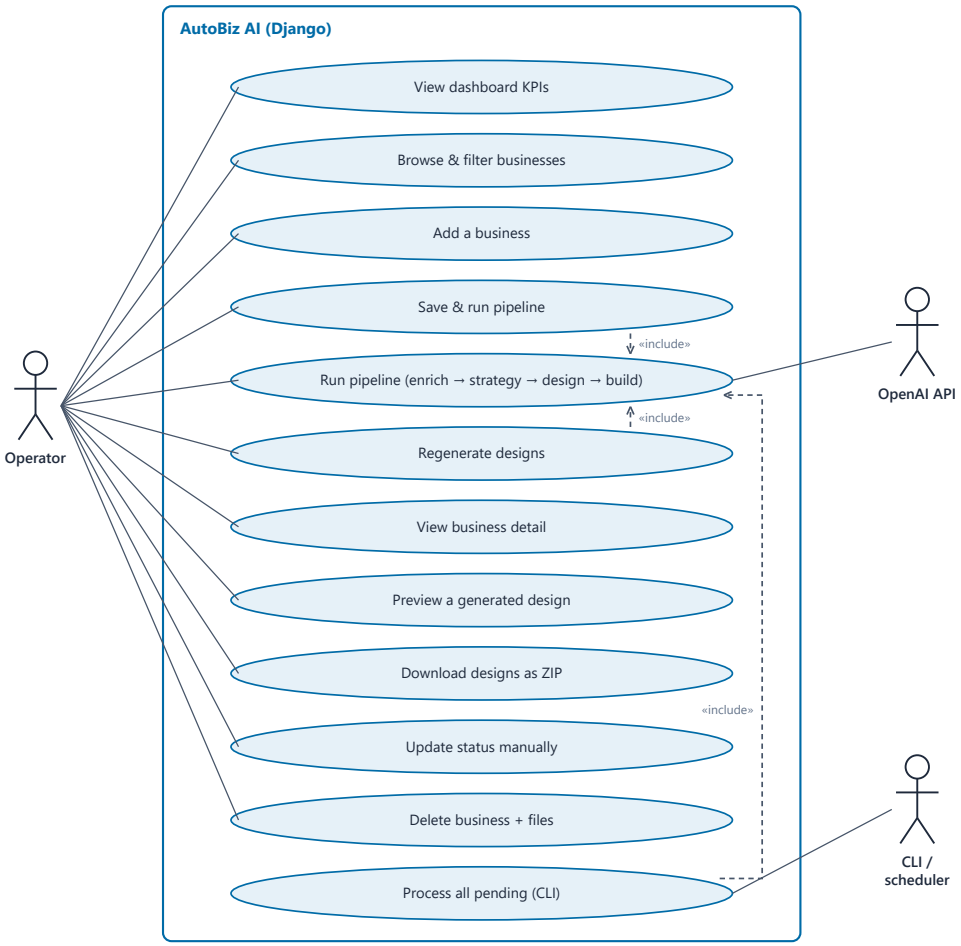


Fig 5.1.2 Use Case Diagram

c) Sequence Diagram

The sequence diagram shows the messages exchanged, in time order, when the operator presses "Run Pipeline": the view calls the Orchestrator, which calls each agent in turn; the agents call the OpenAI API and store their results.

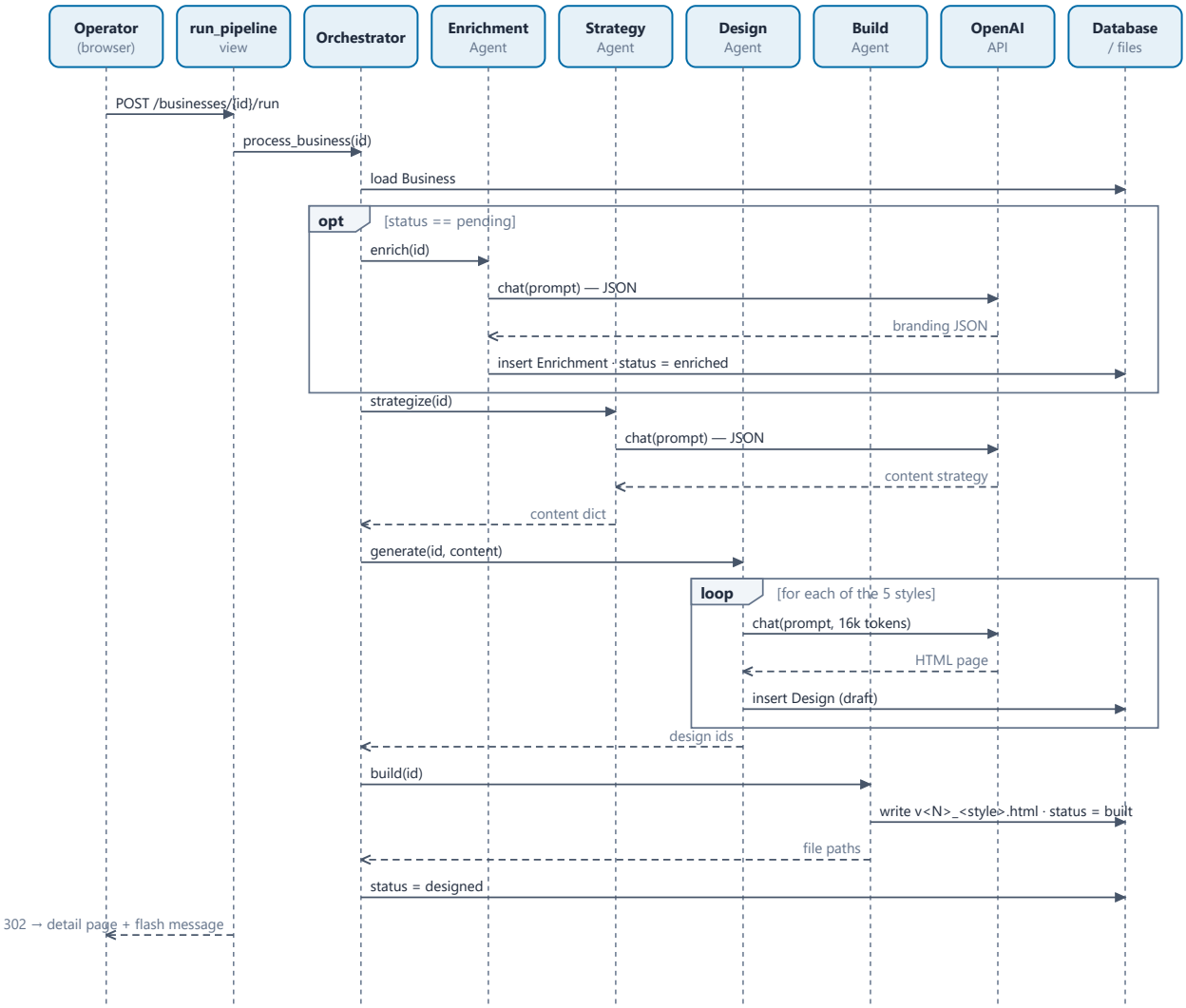


Fig 5.1.3 Sequence Diagram

d) Collaboration Diagram

The collaboration (communication) diagram shows the same interaction as the sequence diagram, organised around the objects and their links. Message numbers give the order; numbers such as 3.1 are messages sent while handling message 3.

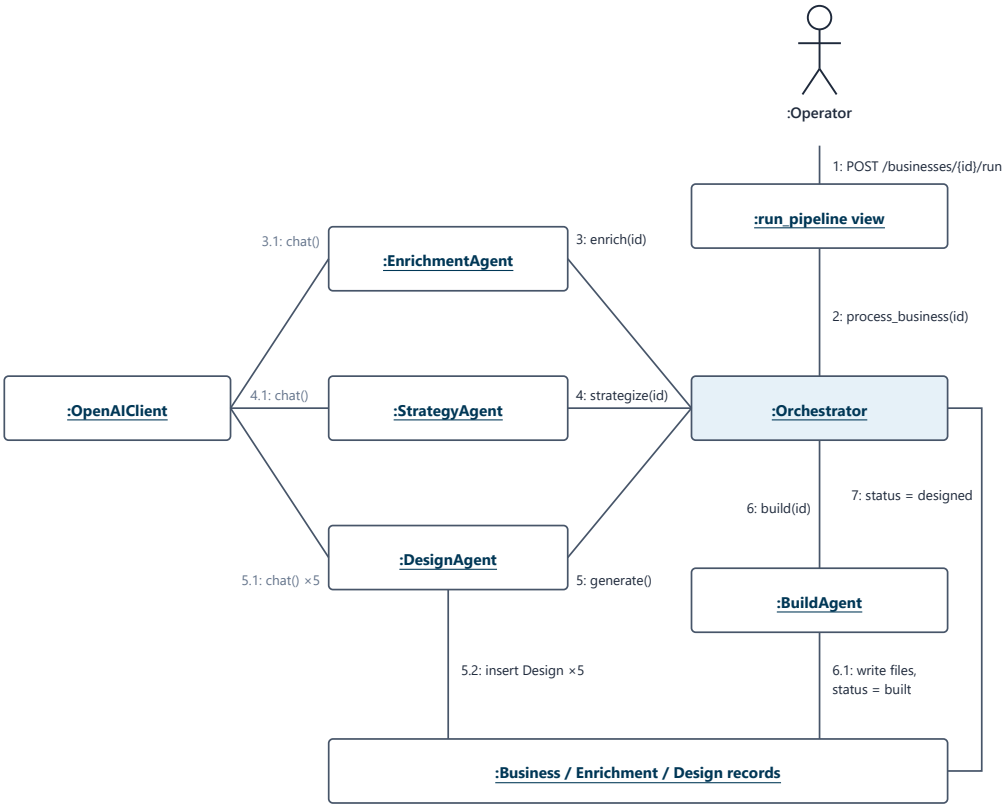


Fig 5.1.4 Collaboration Diagram

e) Component Diagram

The component diagram shows the parts of the system and their dependencies: the Django project with routing, views, templates, services, models and management commands, the storage it uses and the external systems it calls.

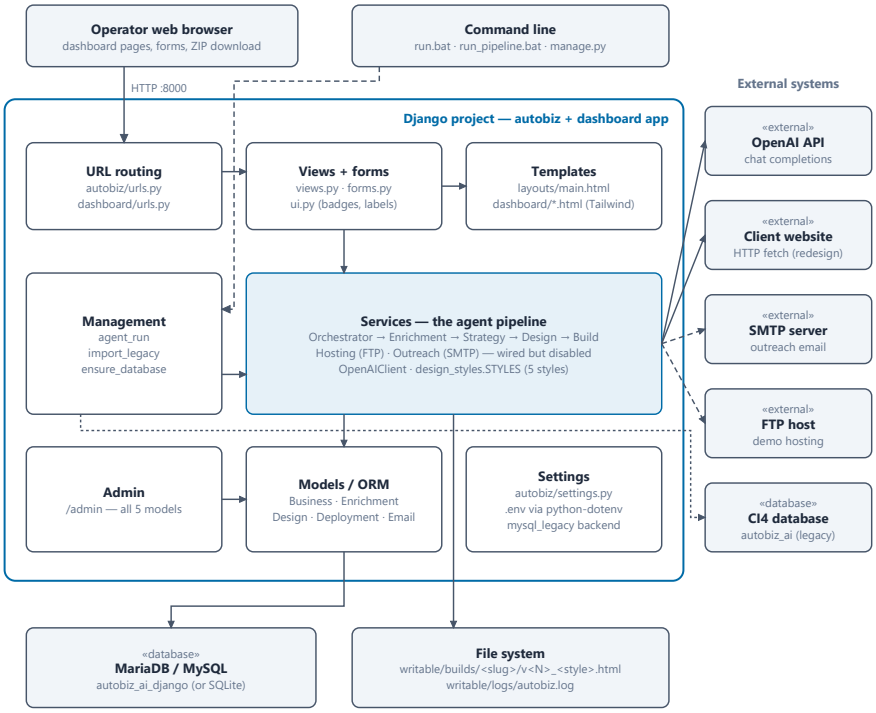


Fig 5.1.5 Component Diagram

f) Deployment Diagram

The deployment diagram shows the physical nodes: the Windows PC that runs the Django server and the XAMPP MariaDB server with the new and the legacy database, and the internet services the system connects to.

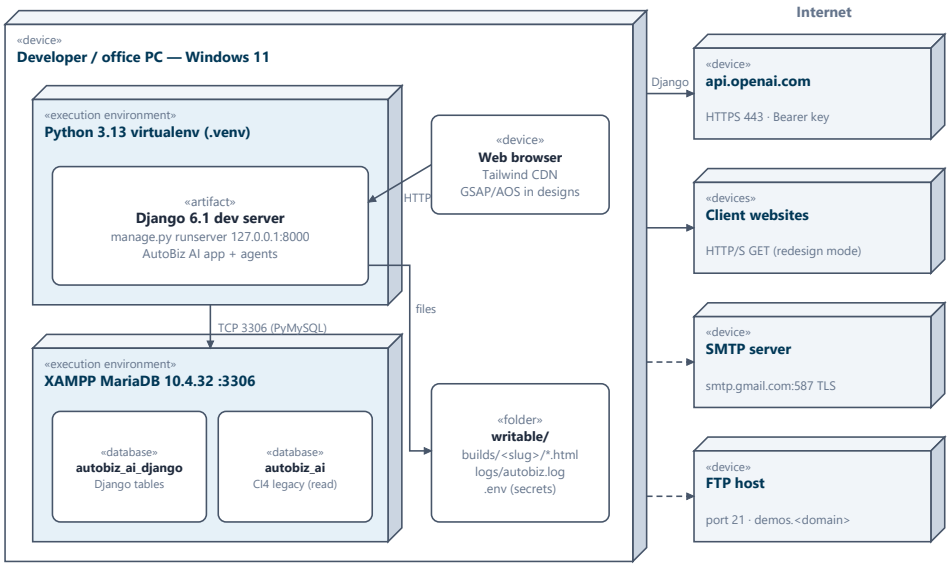


Fig 5.1.6 Deployment Diagram

g) Activity Diagram

The activity diagram shows the control flow of the Orchestrator for one business, including the decisions that skip enrichment for an enriched business and stop the run when a step fails.

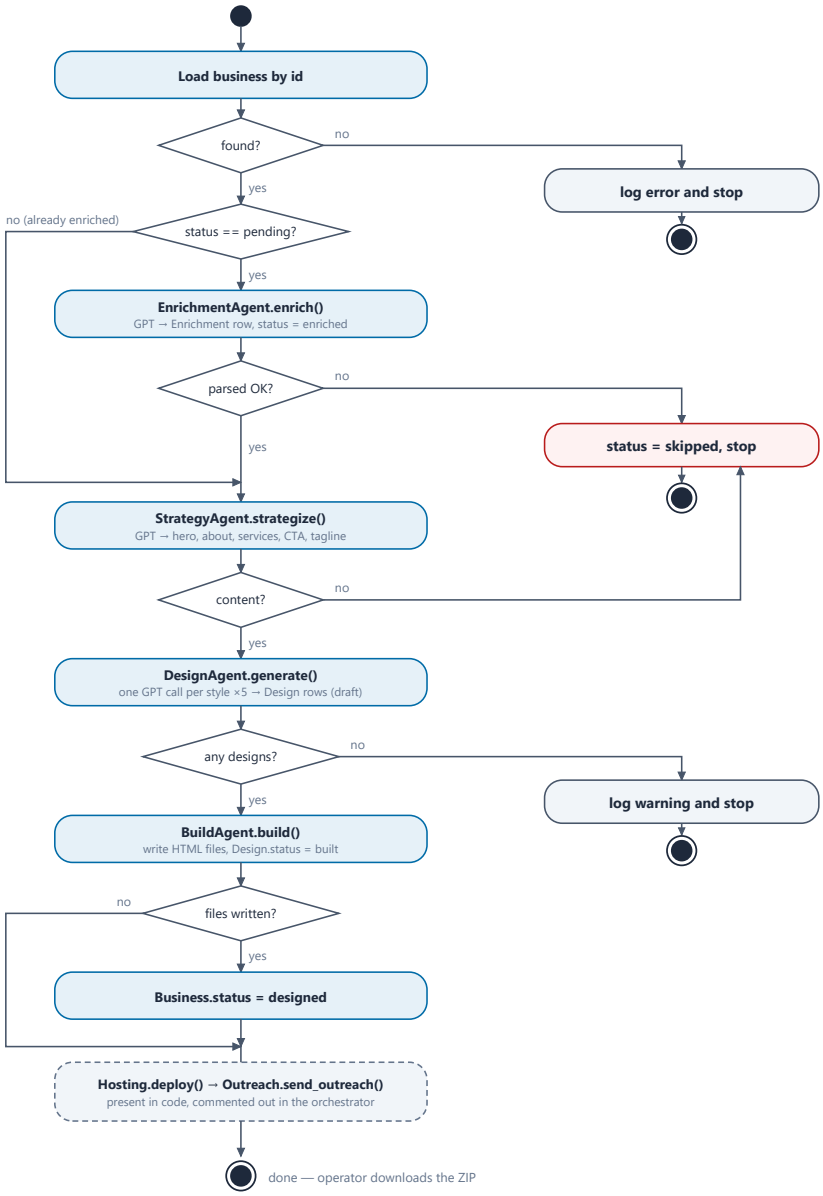


Fig 5.1.7 Activity Diagram

h) Data Flow Diagram

The level-1 data flow diagram shows how data moves between the external entities, the six main processes and the four data stores, from the business details entered by the operator to the finished files.

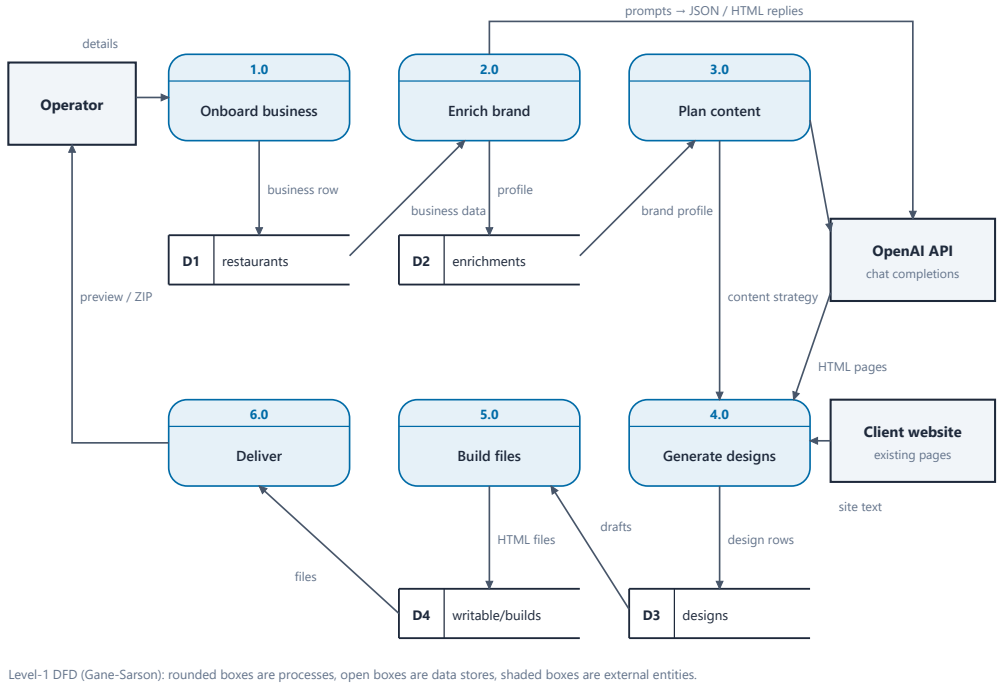


Fig 5.1.8 Data Flow Diagram

Business status life cycle. A new business starts as *pending*. A stored brand profile moves it to *enriched*, and built designs to *designed*; Regenerate moves it back to *enriched*. A failed enrichment or strategy step sets *skipped*. *Deployed* and *contacted* follow when the Hosting and Outreach agents are enabled, and *converted* is set by hand; the manual status form accepts only these seven values.

5.2 Database Design

The database has five tables. Table and column names are identical to the original CodeIgniter migrations, so data can be copied between the two systems without conversion. All foreign keys use ON DELETE CASCADE, and every table has created_at and updated_at timestamps. The status values are listed in the class diagram (Fig 5.1.1).

Table	Primary key / foreign key	Main columns
`restaurants`	id	name, business_type, category, location, address, phone, email, rating, review_count, website_status, website_url, scrape_source, status
`enrichments`	id / restaurant_id	business_summary, target_audience, branding_notes, usp, tone, color_palette, keywords, raw_gpt_response
`designs`	id / restaurant_id	version, style_name, html_code, css_framework, preview_path, status
`deployments`	id / design_id	url, subdomain, ftp_path, status, error_msg, deployed_at
`emails`	id / restaurant_id	to_email, subject, body, demo_links, sent_status, response_status, sent_at, error_msg

CHAPTER - 6
SYSTEM IMPLEMENTATION

6.1 Python and Django

Python is a high-level, general-purpose programming language known for readable syntax, a large standard library and a wide ecosystem of third-party packages. It is the most widely used language for working with AI services, and its "batteries included" approach keeps application code short. AutoBiz AI uses Python 3.13.

Django is a Python web framework that follows the model–template–view pattern. It provides an object-relational mapper (ORM) and migrations for the database, URL routing, form validation, a template language, sessions and messages, protection against cross-site request forgery, an automatic administration interface and a test framework. These built-in parts replace most of the code that had to be written by hand in the PHP version.

Library	Use in the project
Django 6.1.2	Web framework: models, views, templates, forms, admin, tests
PyMySQL 1.2.3	Pure-Python MySQL driver (no C compiler needed on Windows)
requests 2.34	HTTP calls to the OpenAI API and to existing client websites
python-dotenv 1.2	Loads settings and secrets from the .env file
ftplib, zipfile (standard)	FTP upload in the Hosting agent; ZIP download
Tailwind CSS (CDN)	Styling of the dashboard templates

6.2 Algorithms

A. Pipeline orchestration

1. Load the business; stop if it does not exist.
2. If the status is pending, run the Enrichment agent; if its reply cannot be parsed, set the status to skipped and stop.
3. Run the Strategy agent; if it returns no content, set the status to skipped and stop.
4. For each of the five styles, build the design prompt and request an HTML page; keep only replies that contain an HTML document. Stop if no design was produced.
5. Write every draft design to disk and mark it built; if any file was written, set the status to designed.

B. Design prompt construction

The prompt for one style concatenates: the mode (fresh design, or redesign with the fetched title, theme colour and text), the style concept, the font and animation-library links, the CSS foundation the model must paste verbatim, the section-by-section page blueprint, the business data and content strategy, a quality rubric, and a fixed JavaScript block for scroll animations, counters and card tilt. A system message requires raw HTML only and colours through CSS variables only.

C. Existing-site analysis

The page is downloaded (TLS verified first, unverified only after a TLS error). The algorithm extracts the <title>, the theme-color meta tag and the first 600 characters of visible text after removing scripts, styles

and comments. Hex colours found in the page are kept only if their perceived brightness B = (299·R + 587·G + 114·B) / 1000 lies between 30 and 230, which drops near-black and near-white values; at most six are kept.

D. Reply parsing and offline mode

Model replies may be wrapped in Markdown code fences; the parser removes a leading ```json and a trailing ``` and accepts the reply only if it decodes to a JSON object. Without an API key, the client returns stub content chosen by the system message: JSON when the agent asks for JSON, a sample HTML page otherwise.

6.3 Sample Code

The listing below is the core of the system, taken directly from the project source: the Orchestrator method that runs the agent pipeline for one business.

dashboard/services/orchestrator.py — Orchestrator.process_business()

```
    def process_business(self, business_id: int) -> None:
        """Run the full pipeline for a single business."""
        business = Business.objects.filter(pk=business_id).first()
        if not business:
            logger.error('[Orchestrator] Business #%s not found.', business_id)
            return

        logger.info('[Orchestrator] ===== Processing: %s (#%s) =====', business.name, business_id)

        # Step 1: Enrichment
        if business.status == Business.Status.PENDING:
            if not self.enrichment.enrich(business_id):
                logger.warning('[Orchestrator] Enrichment failed for #%s — skipping.', business_id)
                self._set_status(business_id, Business.Status.SKIPPED)
                return

        # Step 2: Content strategy
        content = self.strategy.strategize(business_id)
        if not content:
            logger.warning('[Orchestrator] Strategy failed for #%s — skipping.', business_id)
            self._set_status(business_id, Business.Status.SKIPPED)
            return

        # Step 3: Generate one design per style
        design_ids = self.design.generate(business_id, content)
        if not design_ids:
            logger.warning('[Orchestrator] Design generation failed for #%s.', business_id)
            return
        logger.info('[Orchestrator] Generated %d designs.', len(design_ids))

        # Step 4: Build (write HTML to disk)
        local_paths = self.build.build(business_id)
        logger.info('[Orchestrator] Built %d files.', len(local_paths))

        if local_paths:
            self._set_status(business_id, Business.Status.DESIGNED)

        # Step 5: Host (FTP upload) — disabled; use ZIP download instead
        # demo_urls = self.hosting.deploy(business_id)
        # logger.info('[Orchestrator] Deployed %d pages.', len(demo_urls))

        # Step 6: Outreach email — disabled; notify user via ZIP download
        # if demo_urls:
        #     self.outreach.send_outreach(business_id, demo_urls)

        logger.info('[Orchestrator] ===== Done: %s =====', business.name)
```

CHAPTER - 7
SYSTEM TESTING

Implementation is the stage in which the design becomes a working system, and testing confirms that it does what the requirements say. Because this project is a port, testing had two goals: that every function works, and that the behaviour matches the original PHP application.

7.1 Types of Testing

- **Unit testing** checks single functions in isolation — for example the slug rule for build folders and the parsing of model replies.
- **Integration testing** checks that the agents, the database and the file system work together: the whole pipeline is run with the offline stub and its rows and files are inspected.
- **Functional (system) testing** exercises every route through Django's test client: forms, run, download, preview, regenerate, status change, delete, error pages.
- **Regression and parity testing** compares the port with the original: the five styles, the JavaScript block and the CDN links were compared with the PHP source, and the legacy data was imported.
- **Acceptance testing** runs the system as the operator would: run.bat on the target PC, every page opened in a browser, previews and the ZIP download checked.
- **White-box and black-box testing:** unit and integration tests are written with knowledge of the code (white box); functional and acceptance tests use only inputs and outputs (black box).

7.2 Test Cases

Test Case Id	Test Case Name	Test Case Desc.	Step	Expected	Actual	Status	Priority
01	Add business	Valid form saves and redirects	Submit name, type, city	Business saved as pending, redirect to detail	As expected	Pass	High
02	Form validation	Invalid input is rejected	Name "A", empty city, bad e-mail	Form shown again with errors, nothing saved	As expected	Pass	High
03	Run pipeline	Pipeline produces 5 designs	Run for a pending business	Status designed, 5 built designs and files	As expected	Pass	High
04	Enrichment failure	Bad model reply handled	Model returns non-JSON	Status set to skipped	As expected	Pass	High
05	Batch run	Only pending, no-website businesses	Run agent_run without id	Business with a website untouched	As expected	Pass	Medium
06	Download ZIP	Built designs are zipped	Open /businesses /{id}/download	ZIP with 5 HTML files	As expected	Pass	High
07	Preview	Stored design is served	Open /preview/{id}	HTML page displayed	As expected	Pass	High

Test Case Id	Test Case Name	Test Case Desc.	Step	Expected	Actual	Status	Priority
08	Regenerate	Old designs replaced	Press Regenerate	Old designs deleted, 5 new designs	As expected	Pass	Medium
09	Delete	Rows and files removed	Press Delete	Business, designs and build folder removed	As expected	Pass	High
10	Manual status	Only valid statuses	Post "hacked", then "converted"	First rejected, second saved	As expected	Pass	Medium
11	POST only	State changes need POST	GET the delete URL	HTTP 405	As expected	Pass	Medium
12	Not found	Unknown ids	Open business 999, preview 999	HTTP 404	As expected	Pass	Low
13	Legacy import	Data copied from CI4 database	Run import_legacy	1 business, 1 enrichment, 5 designs, files rebuilt	As expected	Pass	High
14	Style parity	Styles equal the PHP source	Compare via PHP reflection	5 of 5 identical	As expected	Pass	High

Result: all 14 automated tests pass on the XAMPP MariaDB 10.4.32 server (`manage.py test dashboard`). Cases 13 and 14 were run as scripted checks during the port. Not yet verified: a live run against the OpenAI API, and real SMTP and FTP transfers, which are disabled in the pipeline.

CHAPTER - 8
SCREENSHOTS

To run the project, install Python 3.13 and XAMPP, and start MySQL in the XAMPP control panel. Then double-click **run.bat** in the project folder. On the first run it creates a virtual environment and installs the packages from requirements.txt; every run creates the database if needed, applies the migrations and starts the web server, then opens the browser at http://127.0.0.1:8000.

```
run.bat — console output
```

```
[1/5] Virtual environment found.
[2/5] Installing requirements...
[3/5] Ensuring database exists...
  MySQL database `autobiz_ai_django` is ready.
[4/5] Applying migrations...
  No migrations to apply.
[5/5] Starting server at http://127.0.0.1:8000  (Ctrl+C to stop)
```

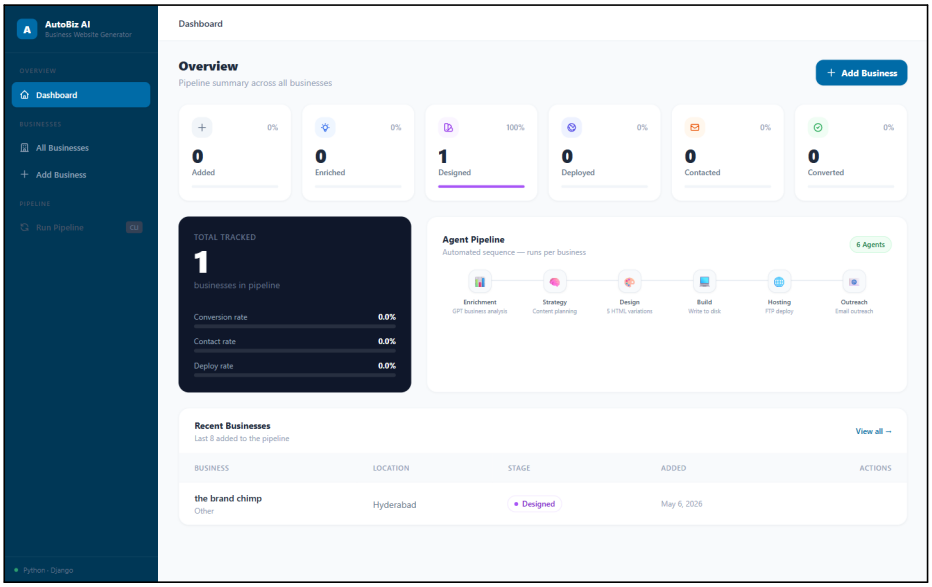


Fig 8.1 Dashboard

In above screen the dashboard shows how many businesses are in each pipeline stage, the conversion, contact and deploy rates, the six agents and the most recent businesses. Click "All Businesses" in the menu to get below page.

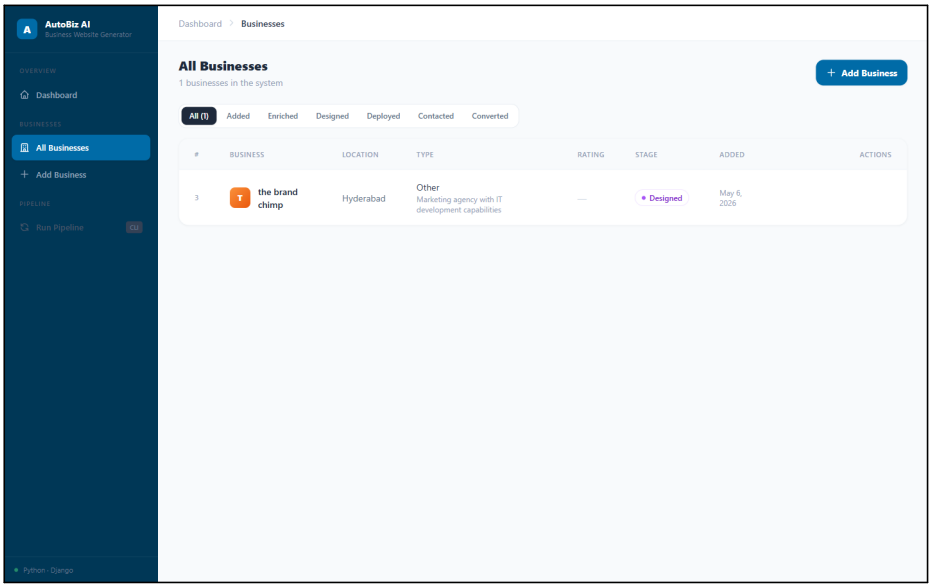


Fig 8.2 All Businesses

In above screen all businesses are listed with their location, type, rating and stage. The tabs filter by stage, and hovering a row shows the View, Run and Delete buttons. Click "Add Business" to get below page.

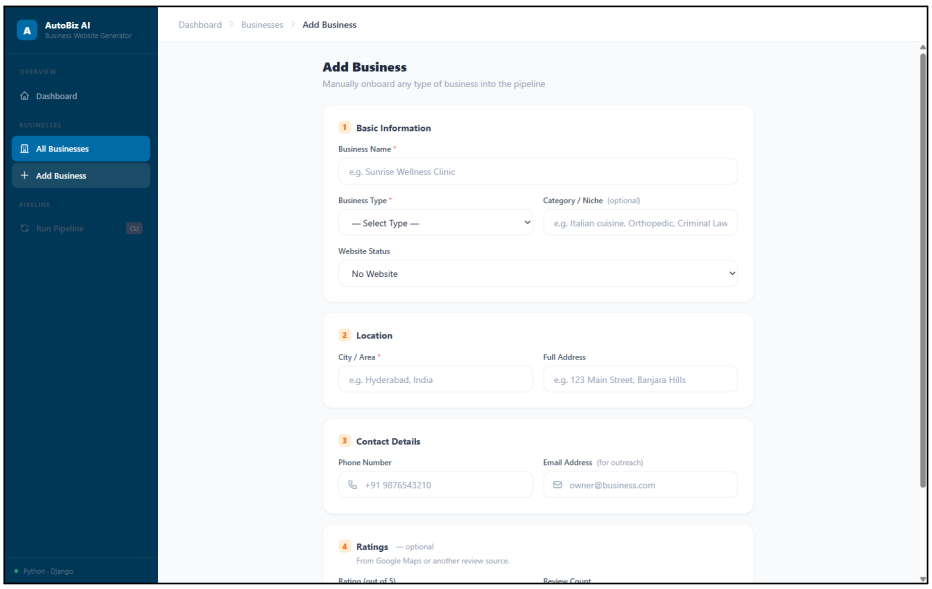


Fig 8.3 Add Business Form

In above screen the operator enters the business details in four sections. "Save Business" stores it; "Save & Run Pipeline" stores it and generates the designs immediately. If the input is invalid the form is shown again with messages, as in below screen.

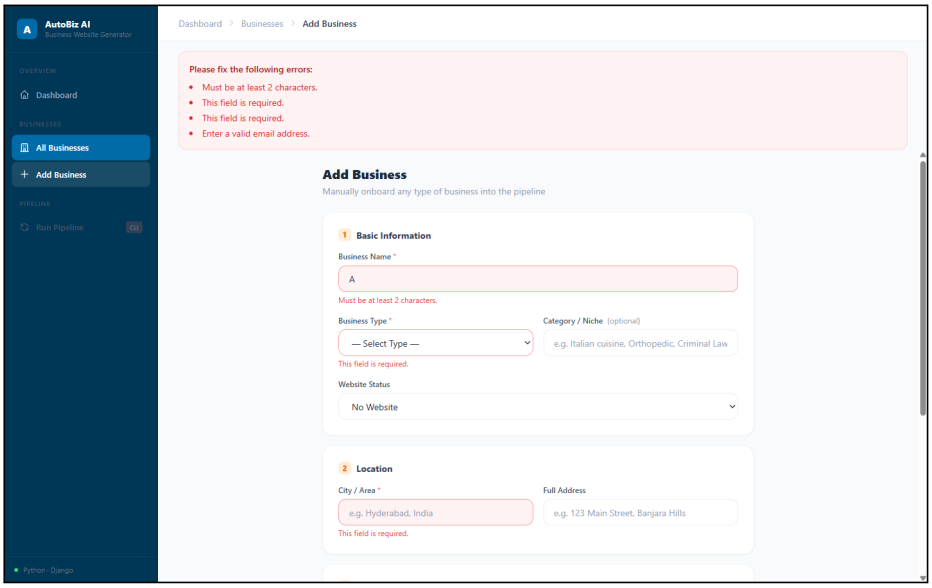


Fig 8.4 Form Validation Errors

In above screen the name is too short, the type and city are missing and the e-mail address is invalid; each field is highlighted with its own message and nothing is saved.

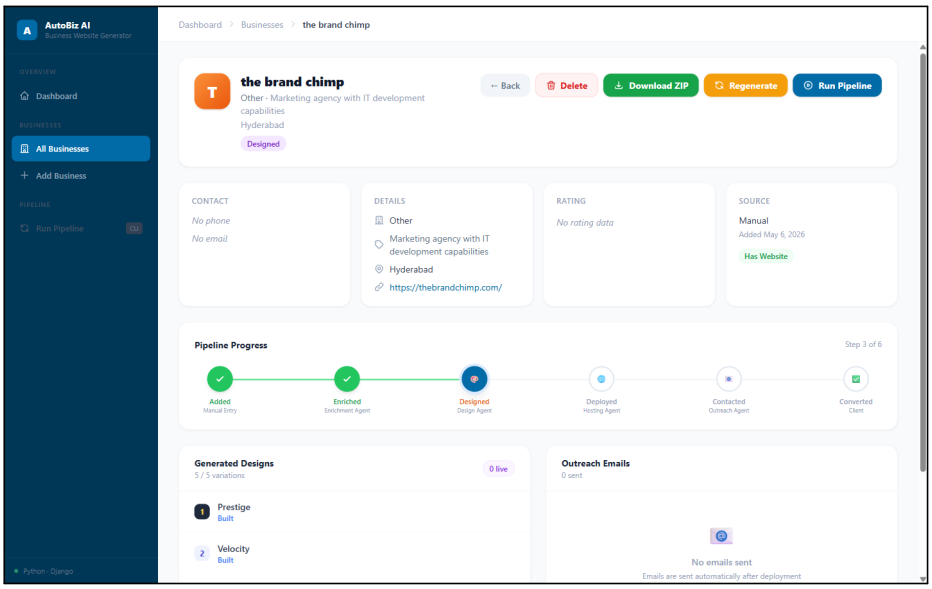


Fig 8.5 Business Detail

In above screen the business detail page shows the contact and source information, the pipeline progress and the five generated designs. The buttons run the pipeline, regenerate the designs, download the ZIP file or delete the business. Clicking "Preview" on a design opens it, as in below screens.

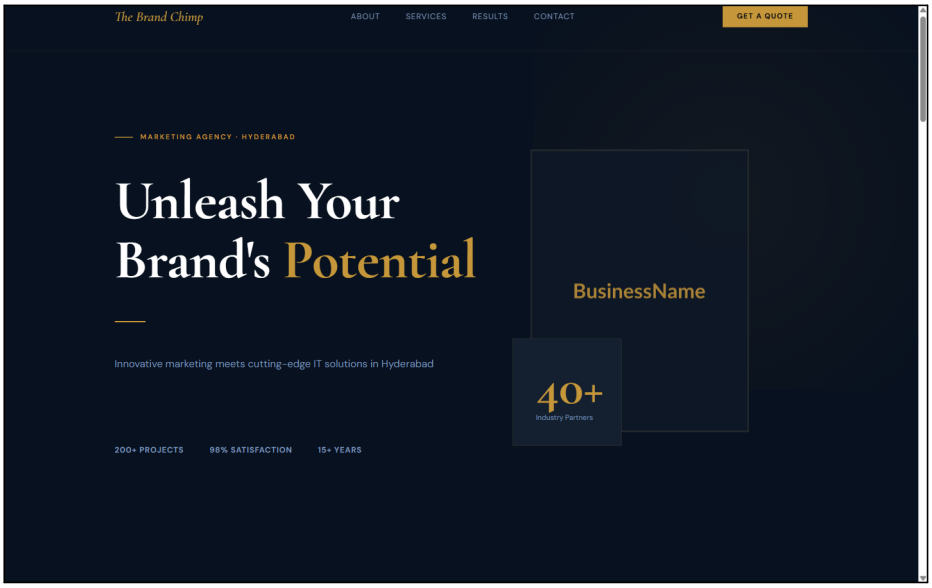


Fig 8.6 Generated Design — Prestige Style

In above screen the Prestige design for the business uses a dark navy background with champagne-gold accents, a serif headline built from the generated copy and an animated statistics card.

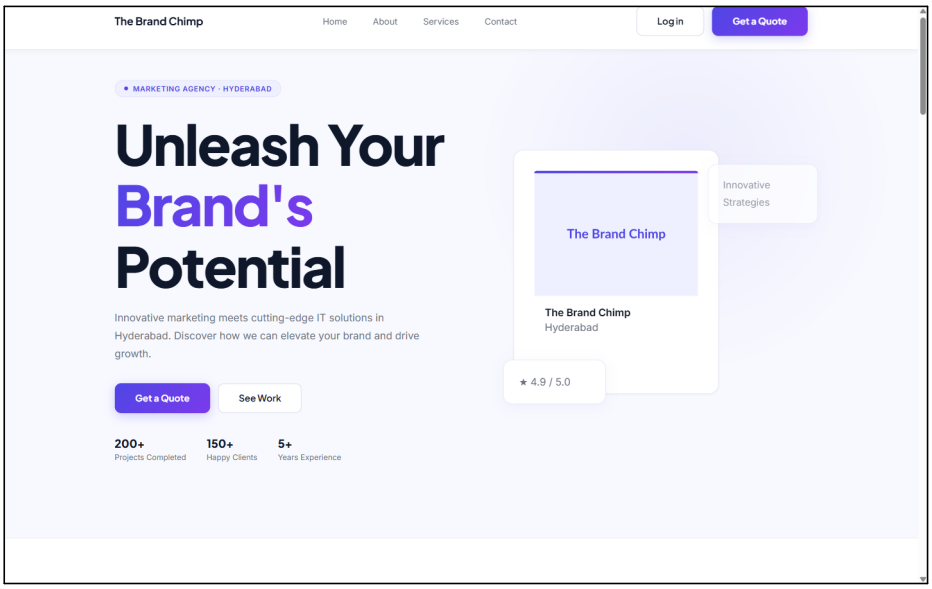


Fig 8.7 Generated Design — Velocity Style

In above screen the Velocity design for the same business uses a light layout with an indigo-to-purple gradient. Similarly, the Canvas, Obsidian and Editorial designs can be previewed, and all five can be downloaded together as a ZIP file.

CHAPTER - 9
CONCLUSION

This project shows that a pipeline of specialised language-model agents can turn a handful of facts about a business into several complete, professional website designs in a few minutes. Splitting the work into enrichment, strategy, design and build steps, each with a precise prompt and a stored result, keeps the output consistent and makes every step repeatable; five fixed visual styles with verbatim CSS foundations give the owner real choice while keeping quality predictable.

The re-implementation in Python and Django achieved its goals. The database schema, prompts, design styles and user interface are unchanged, and the original data was migrated with one command. At the same time the system became easier to install (one double-click), safer (validated input, CSRF protection, POST-only changes) and verifiable (14 automated tests that pass on the target MariaDB 10.4 server). Several defects of the original were fixed on the way, including the offline mode that stopped at the strategy step.

AutoBiz AI is therefore ready for local use by an operator, and its modular agent design leaves a clear path for the enhancements described in the next chapter.

CHAPTER - 10
FUTURE ENHANCEMENTS

The current system provides a solid base for real-world use. The following enhancements are planned, with rough effort estimates:

1. **Background processing (1–2 days).** Run the pipeline in a task queue (Django-Q2, Celery or RQ) with live progress, so the browser no longer waits 1–3 minutes for a request.
2. **Authentication (½ day).** Require a login for the dashboard so that only authorised users can start paid API calls.
3. **Production deployment (1 day).** Serve with waitress or gunicorn behind IIS or nginx, with DEBUG off and a supported database (MariaDB 10.11+ or MySQL 8.4).
4. **Live hosting and outreach (1 day).** Re-enable the Hosting and Outreach agents and test FTP upload and e-mail delivery end to end.
5. **Lead discovery (2–3 days).** A scraping agent (the project already has an Apify token configured) that finds businesses without websites and adds them automatically.
6. **Static front-end build (½ day).** Compile Tailwind CSS into static files instead of loading it from a CDN.
7. **Design editing and selection.** Let the owner pick a design, edit texts and images in place and publish the selected site.
8. **More styles and languages.** Additional visual styles per industry and generation of the website copy in regional languages.

CHAPTER - 11
REFERENCES

[1] T. Brown et al., "Language Models are Few-Shot Learners", Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[2] M. Chen et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, 2021.

[3] T. Beltramelli, "pix2code: Generating Code from a Graphical User Interface Screenshot", Proc. ACM SIGCHI Symposium on Engineering Interactive Computing Systems, 2018 (arXiv:1705.07962, 2017).

[4] C. Si, Y. Zhang, Z. Yang, R. Liu and D. Yang, "Design2Code: How Far Are We From Automating Front-End Engineering?", arXiv:2403.03163, 2024.

[5] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models", International Conference on Learning Representations (ICLR), 2023.

[6] C. Qian et al., "ChatDev: Communicative Agents for Software Development", arXiv:2307.07924, 2023.

[7] S. Hong et al., "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework", arXiv:2308.00352, 2023.

[8] OpenAI, "GPT-4 Technical Report", arXiv:2303.08774, 2023.

[9] Django Software Foundation, Django documentation, https://docs.djangoproject.com/

[10] OpenAI, API reference — Chat Completions, https://platform.openai.com/docs/api-reference/chat

[11] CodeIgniter Foundation, CodeIgniter 4 User Guide, https://codeigniter.com/user_guide/

[12] MariaDB Foundation, MariaDB Server documentation, https://mariadb.com/kb/en/documentation/

[13] PyMySQL contributors, PyMySQL documentation, https://pymysql.readthedocs.io/

[14] Object Management Group, OMG Unified Modeling Language (UML) Specification, version 2.5.1, 2017.

[15] C. Gane and T. Sarson, Structured Systems Analysis: Tools and Techniques, Prentice Hall, 1979.

[16] ReportLab, ReportLab PDF library user guide, https://docs.reportlab.com/